FINAL SUBMISSION

# DATA 266 - Homework 1

DATA 266 | HOMEWORK 1

# Neural networks, framework comparison, and CUDA matrix multiplication


## Kavan Siddesh Kumar

SID4 = 9486 | SEED = 9486 | SLICE = 486

HP_ID = 0 | CLS_A = 6 | CLS_B = 0


### HP_ID configuration

| Hidden layers | Learning rate | Epochs |
|---|---|---|
| [32] | 0.001 | 30 |

This report consolidates verified repository results for autoregressive models, diabetes preprocessing and visualization, PyTorch and TensorFlow/Keras experiments, and the executed CUDA benchmark and profiler run. AI use is disclosed separately in AI_USE.md.

# Autoregressive Models and Dataset

DATA 266 | HOMEWORK 1

## 1. Autoregressive models

An autoregressive (AR) model predicts the current or future value of a time series from previous values of that same series. Those observations are lagged values. An AR(p) model uses the previous p values:

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \varepsilon_t$$

Here, y_t is predicted, c is a constant, phi terms are learned coefficients, epsilon_t is unpredictable error, and p is the number of lags. An AR(2) temperature model uses yesterday and two-days-ago temperatures. Other uses include electricity demand, sales, financial values, and weather measurements. AR models are simple, computationally efficient, and interpretable, but work best when mean and variance are reasonably stationary.

# Dataset Visualization

DATA 266 | HOMEWORK 1

## Verified correlations

Glucose has the strongest reported linear association, followed by BMI, Insulin, and SkinThickness. The existing repository figures are embedded directly.

These Pearson correlations with the remapped Diabetes target are descriptive and do not prove causation or statistical significance.

| Feature | Correlation |
|---|---|
| Glucose | 0.491538 |
| BMI | 0.315051 |
| Insulin | 0.303797 |
| SkinThickness | 0.262079 |
| Pregnancies | 0.218405 |
| BloodPressure | 0.169221 |
| DiabetesPedigreeFunction | 0.163246 |
| Age | 0.112565 |

FIGURE 1

# Correlation Matrix

DATA 266 | HOMEWORK 1

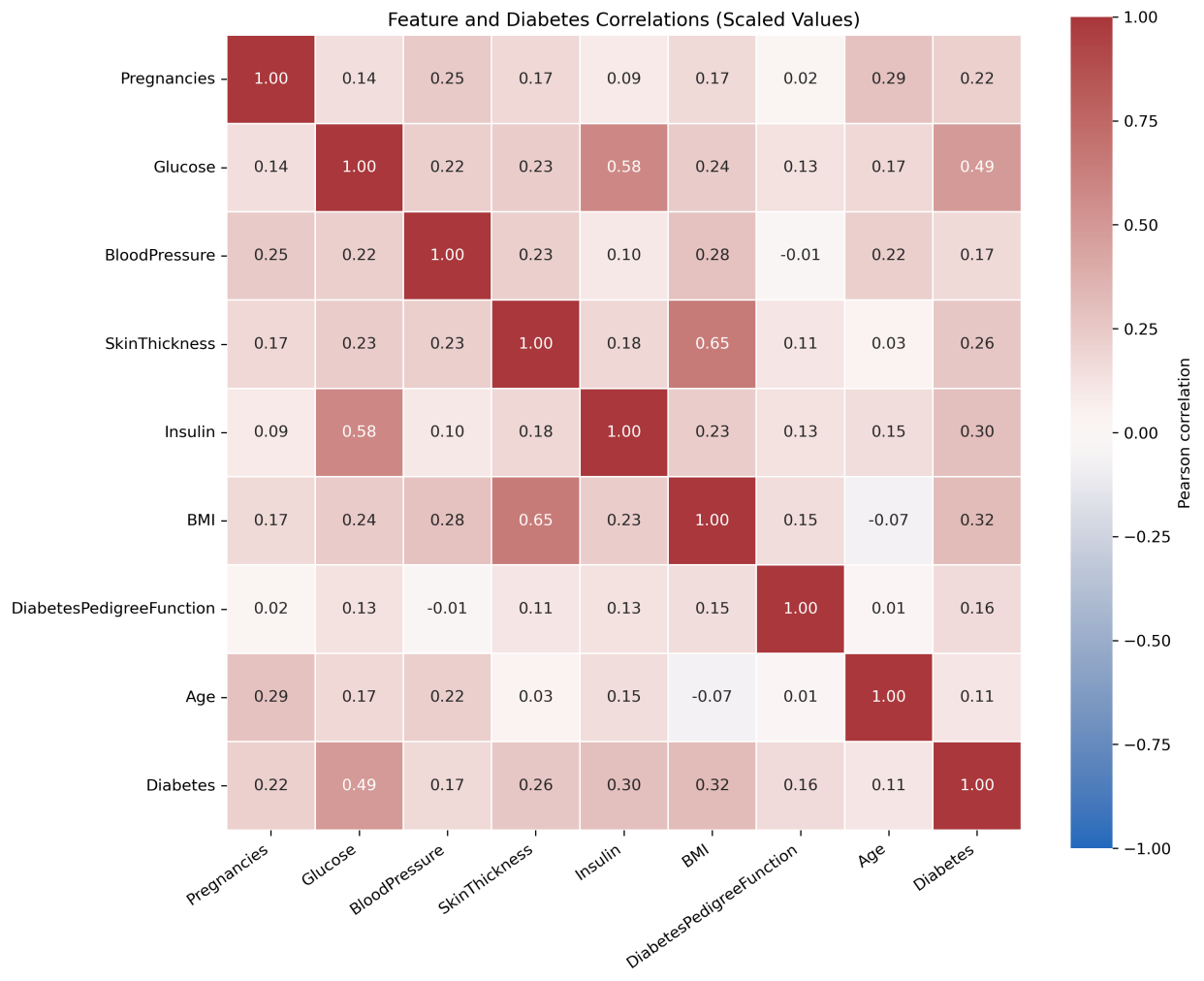


*Figure 1. Existing tracked correlation matrix embedded directly.*

FIGURE 2

# Feature Distributions

DATA 266 | HOMEWORK 1

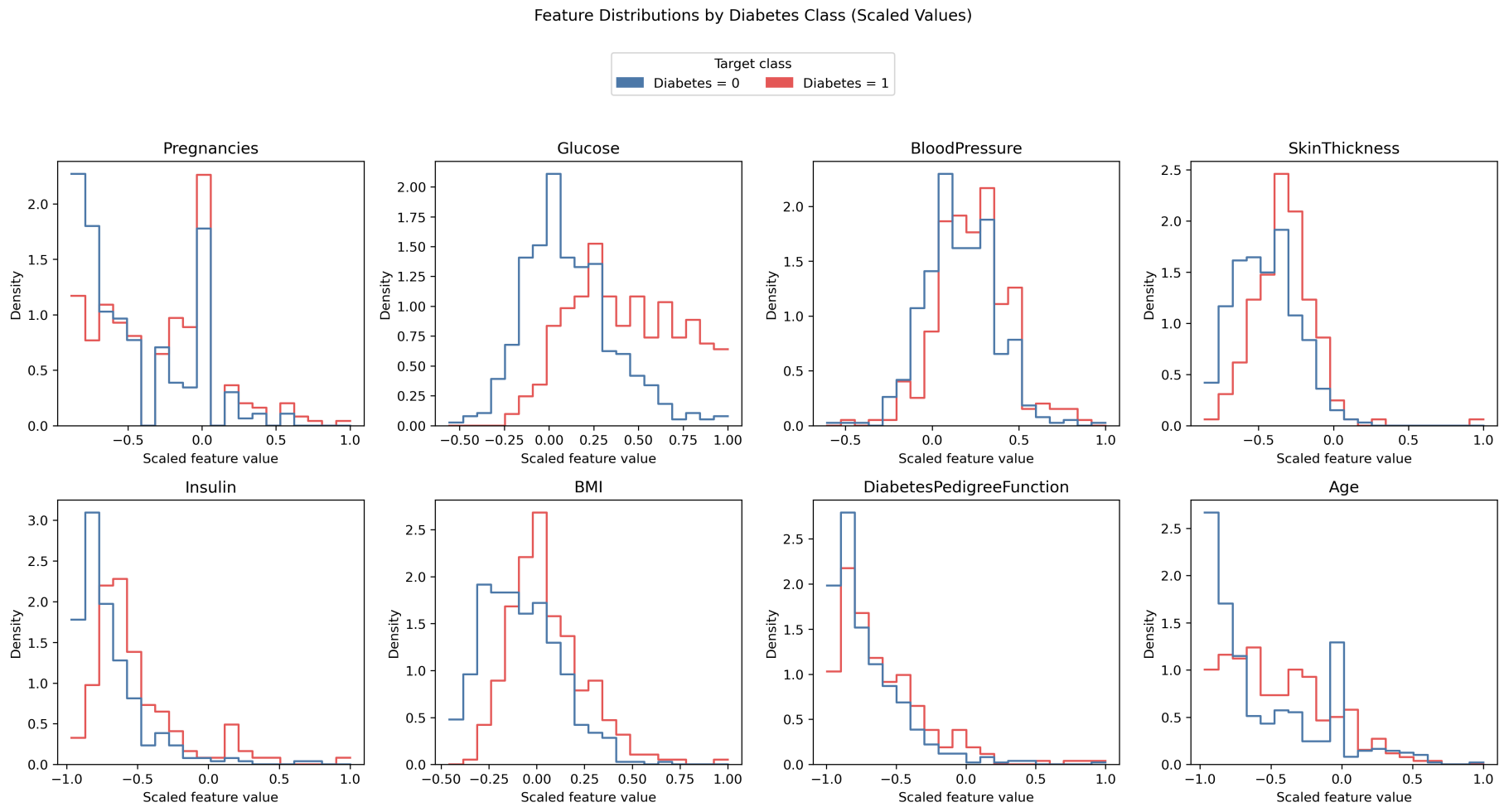


*Figure 2. Existing tracked feature distributions embedded directly.*

# Neural-Network Configurations and PyTorch

DATA 266 | HOMEWORK 1

## Configuration comparison

| Model | Hidden layers | Learning rate | Epochs | Parameters |
|---|---|---|---|---|
| Baseline | [64, 32] | 0.001 | 30 | 2,689 |
| Modified (HP_ID 0) | [32] | 0.001 | 30 | 321 |

Both frameworks use the same fixed split, preprocessing, seeds 9486/9487/9488, batch size 32, and test-accuracy metric. PyTorch uses nn.Module, DataLoader, and a manual loop. It uses raw logits with BCEWithLogitsLoss and applies sigmoid for prediction, mathematically equivalent to sigmoid plus binary cross-entropy.

# PyTorch Loss Curves

DATA 266 | HOMEWORK 1

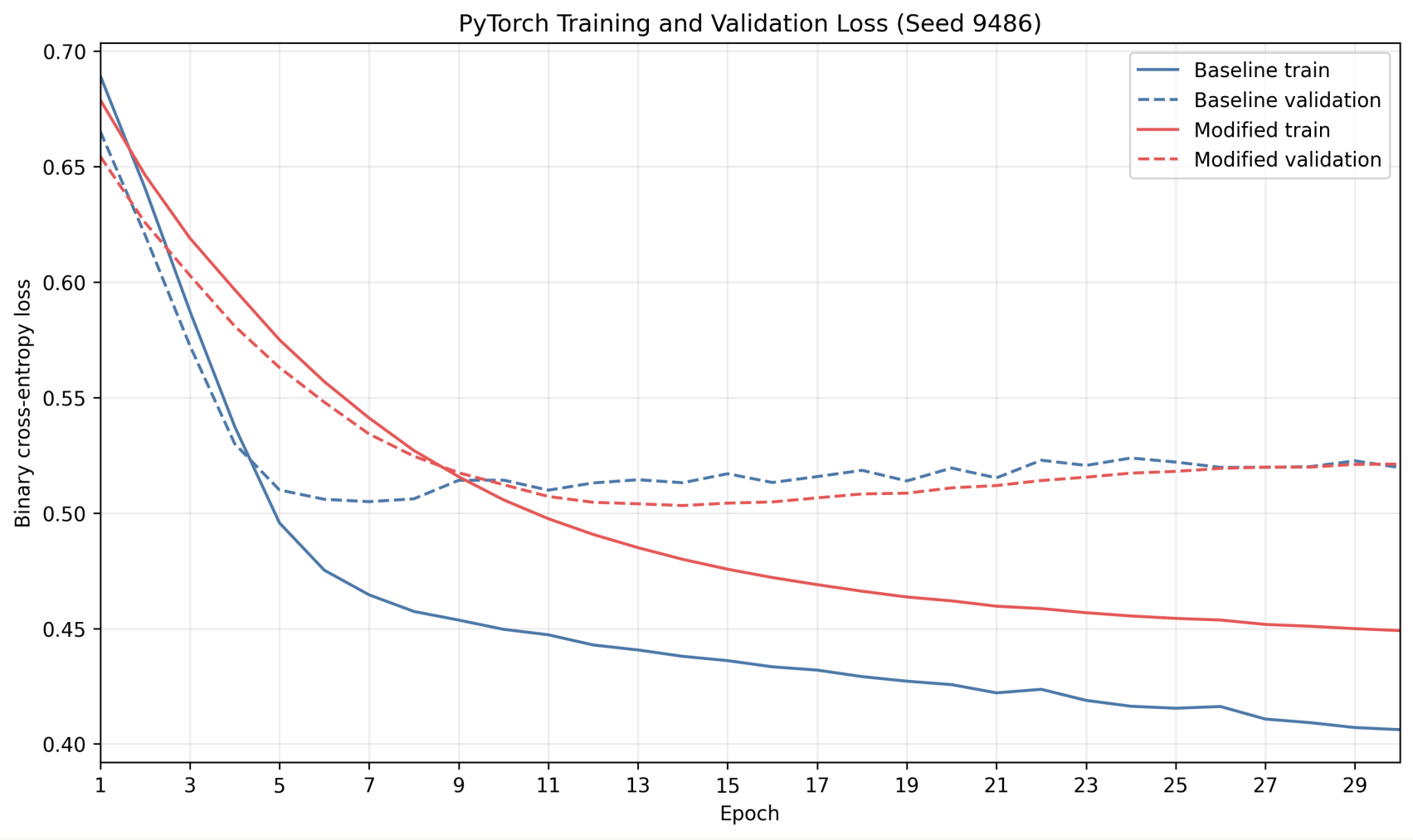


*Figure 3. Executed PyTorch training and validation loss curves for seed 9486.*

Minimum validation-loss epochs: baseline 7, modified 14. Final validation-training gaps: approximately 0.1135 and 0.0721.

# TensorFlow/Keras Results

DATA 266 | HOMEWORK 1

## Class-demo implementation

TensorFlow/Keras follows the class demonstration with Sequential and Dense layers, sigmoid output, binary_crossentropy, metrics=[accuracy], model.fit(), model.evaluate(), and model.predict(). Evaluated accuracy matched manually thresholded predictions.



## Seed-level results

COMPARISON

# TensorFlow Curves and Comparison

DATA 266 | HOMEWORK 1

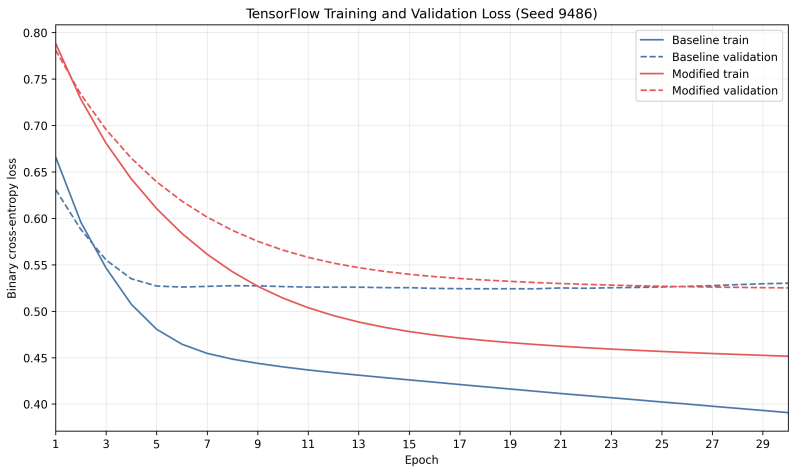


*Figure 4. TensorFlow/Keras loss curves for seed 9486.*

## Framework summary

| Framework | Model | Mean | Sample SD | Params |
|---|---|---|---|---|
| PyTorch | Baseline | 79.24% | 0.5064 pp | 2,689 |
| PyTorch | Modified | 80.41% | 1.0129 pp | 321 |
| TensorFlow | Baseline | 79.53% | 3.3210 pp | 2,689 |
| TensorFlow | Modified | 80.70% | 0.8772 pp | 321 |



Both frameworks improved by approximately 1.17 percentage points after reducing parameters from 2,689

# CUDA Benchmark and Conclusion

DATA 266 | HOMEWORK 1

## Unprofiled benchmark

| Size | CPU ms | Kernel ms | H2D+D2H ms | GPU total ms | Speedup |
|---|---|---|---|---|---|
| 256 | 0.510085 | 0.113312 | 0.258176 | 0.371488 | 1.373086 |
| 1024 | 19.007706 | 4.370048 | 2.917920 | 7.287968 | 2.608094 |
| 4096 | 1208.051661 | 194.599136 | 54.218529 | 248.817665 | 4.855168 |

The CUDA program uses TILE_SIZE=16, 16 x 16 blocks, one thread per output element, shared-memory tiling, boundary zero-padding, OpenBLAS cblas_sgemm, warm-up, and median separated timings. Every size passed abs_error <= 1e-3 + 1e-3 * abs(cpu_value). The GPU was faster at every tested size; 256 x 256 is the smallest tested beneficial size, so crossover is only established at or below 256, not the exact sub-256 crossover.